

What is JavaScript?

JavaScript is a programming language used to make websites interactive and dynamic. It was created by **Brendan Eich** in 1995 while he was working at Netscape. JavaScript is one of the core technologies of the web, along with **HTML** and **CSS**.

It runs in all modern web browsers like Chrome, Firefox, Edge, Safari, etc. JavaScript is a **client-side language**, which means it runs on the user's browser, not on the server (by default).

Using JavaScript, we can change the content of a webpage, respond to user actions like clicks, form input, and create animations, sliders, pop-ups, and much more.

@Mastercode.sagar

Simple Example:



← When we click the button, the text changes.

Example Code (HTML + JavaScript):

```
<button onclick = "showMsg()" > Click Me </button>
<p id = "msg" > </p>
<script>
  function showMsg() {
    document.getElementById("msg").innerText = "Hello World!";
  }
</script>
```

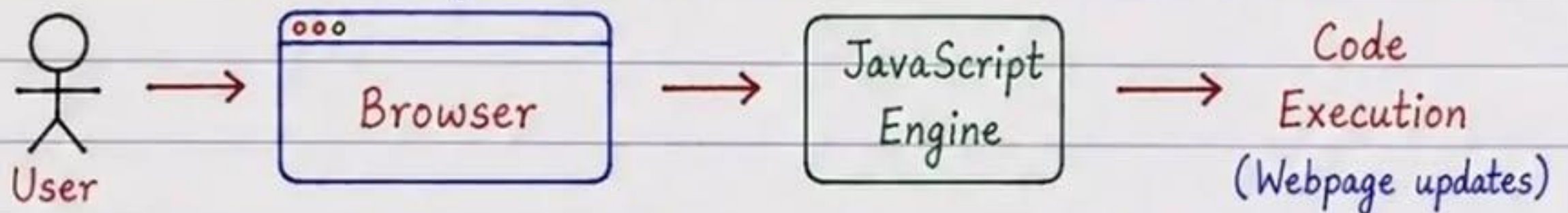
← This JavaScript code changes the text when the button is clicked.

How JavaScript Works (Browser & Node.js)

JavaScript can run in two main environments:
Web Browsers and Node.js (server-side).

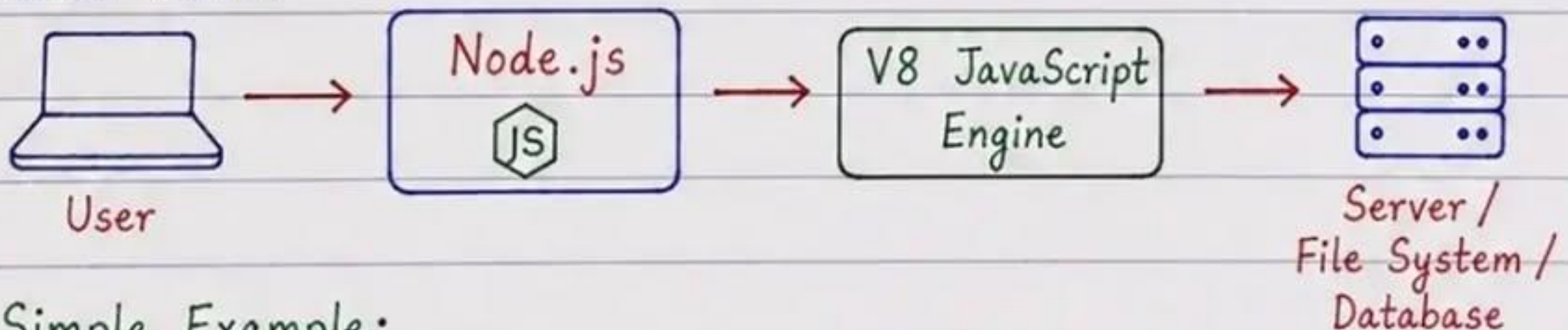
1. JavaScript in the Browser

When we open a website, the browser loads the **HTML**, **CSS**, and **JavaScript** files. The JavaScript code is then read and executed by the browser's JavaScript Engine. This engine understands the code and makes the webpage interactive and dynamic.



2. JavaScript in Node.js

Node.js allows JavaScript to run outside the browser, on the server. It uses the V8 JavaScript Engine (the same engine used in Chrome). This lets us run JavaScript to build backend services, handle files, databases, and much more.



Simple Example:

Browser: When you click a button on a webpage and the text changes without reloading – that's JavaScript working in the browser.

Node.js: When you run a JavaScript file using '`node app.js`' in the terminal and it prints data or handles a file – that's JavaScript running on the server with Node.js.

Variables (let, const, var)

Variables are containers used to store data values.

In JavaScript, we can declare variables using **let**, **const**, or **var**.

1. var (Old way - function scoped)

- Declared using **var**.
- Function scoped or globally scoped.
- Can be **re-declared** and updated.
- Hoisted (can be used before declaration, value is **undefined**).

Example:

```
var name = "Akash";  
window var name = "Akash";  
name = "Bangare";  
// updated  
console.log(name); // Bangare
```

2. let (Modern way - block scoped)

- Declared using **let**.
- Block scoped { }.
- Can be updated but cannot be **re-declared** in the same scope.
- Hoisted but not initialized, so cannot be used before declaration. (**ReferenceError**)

Example:

```
let age = 20;  
age = 21; // updated  
{  
  let age = 22; // block scoped  
  console.log(age); // 22  
}  
console.log(age); // 21
```

3. const (Modern way - block scoped)

- Declared using **const**.
- Block scoped { }.
- Cannot be **re-assigned** or **re-declared**.
- Must be initialized at the time of declaration.

Example:

```
const pi = 3.14;  
// pi = 3.141; // Error  
  
const user = { name: "Akash" };  
user.name = "Bangare"; // Allowed  
console.log(user.name); // Bangare
```

Quick Comparison:

Feature	var	let	const
Scope	Function / Global	Block	Block
Re-declare	Yes	No	No
Re-assign	Yes	Yes	No
Hoisting	Yes (undefined)	Yes (not initialized)	Yes (not initialized)
Initialization Required	No	No	Yes

Data Types (Primitive)

Data types define the kind of values a variable can hold.

In JavaScript, **primitive data types** store single values.

They are the most basic building blocks of data.

There are 7 primitive data types in JavaScript.

Type	Description	Example	Code Example
① String	Represents text enclosed in quotes.	"Hello, JavaScript!"	let msg = "Hello, JS!";
② Number	Represents numbers, both integers and decimals.	42 , 3.14	let age = 25; let pi = 3.14;
③ Boolean	Represents true or false .	true , false	let isStudent = true;
④ Undefined	A variable that has been declared but not assigned a value.	undefined	let x; console.log(x); // undefined
⑤ Null	Represents the intentional absence of any value.	null	let data = null;
⑥ Symbol	Represents a unique and immutable value.	Symbol('id')	let id = Symbol('id');
⑦ BigInt	Represents integers larger than the safe limit of Number.	9007199254740991n	let big = 9007199254740991n;

Examples in Use:

let name = "Akash";

// String

let age = 25;

// Number

let isActive = true;

// Boolean

let score;

// Undefined

let value = null;

// Null

let uniqueId = Symbol('id');

// Symbol

let bigNumber = 9007199254740991n;

// BigInt

Non-Primitive (Reference) Data Types

Type	Description	Example	Code Example
① Object	A collection of key-value pairs.	<pre>{ name: "Akash", age: 25, isStudent: true }</pre>	<pre>let user = { name: "Akash", age: 25, isStudent: true };</pre>
② Array	An ordered list of values.	<pre>[10, 20, 30]</pre>	<pre>let numbers = [10, 20, 30];</pre>
③ Function	A block of code designed to perform a task.	<pre>function greet() { console.log("Hello!"); }</pre>	<pre>function greet() { console.log("Hello!"); } greet();</pre>

Type Conversion

Type conversion in JavaScript is the process of converting one data type into another.

It can happen automatically by JavaScript or manually by us.

1. Implicit Type Conversion (Automatic)

- JavaScript automatically converts types when needed.
- This happens in expressions and operations.

Example:

```
let a = "5" + 2;    // "5" (string) is converted to number 5
console.log(a);     // Output: "52" (because + with string does concatenation)

let b = "5" - 2;    // "5" (string) is converted to number 5
console.log(b);     // Output: 3
```

2. Explicit Type Conversion (Manual)

- We can explicitly convert data types using built-in methods.
- Common methods:

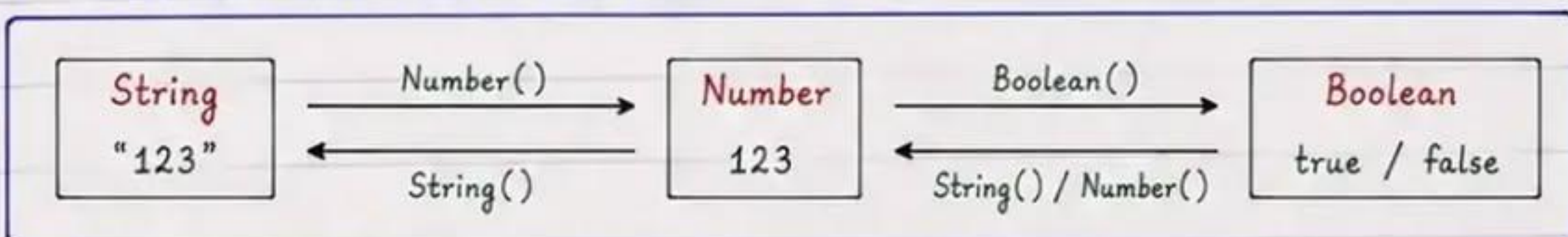
Method	Converts To	Example	Result
String()	String	String(123)	"123"
Number()	Number	Number("45")	45
Boolean()	Boolean	Boolean(0)	false
parseInt()	Integer Number	parseInt("56px")	56
parseFloat()	Decimal Number	parseFloat("3.14abc")	3.14

Examples:

let s = String(100); console.log(s); // Output: "100"	let n = Number("78"); console.log(n); // Output: 78	let b = Boolean(1); console.log(b); // Output: true	let x = parseInt("123.9"); console.log(x); // Output: 123
---	---	---	---

3. Why Type Conversion is Important?

- Helps in performing operations between different data types.
- Useful when taking user input (which is usually a string).
- Makes our code more flexible and error-free.



Operators

Operators are special symbols used to perform operations on variables and values.

JavaScript has different types of operators.

They help us do math, compare values, assign values, and more.

1. Arithmetic Operators (for math operations)

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 2	5
%	Modulus (Remainder)	10 % 3	1
**	Exponentiation	2 ** 3	8

2. Assignment Operators (for assigning values)

Operator	Name	Example	Result
=	Assign	let x = 5	x = 5
+=	Add and assign	x += 2	x = x + 2
-=	Subtract and assign	x -= 2	x = x - 2
*=	Multiply and assign	x *= 2	x = x * 2
/=	Divide and assign	x /= 2	x = x / 2
%=	Modulus and assign	x %= 2	x = x % 2

3. Comparison Operators (for comparing values)

Operator	Name	Example	Result
==	Equal to	5 == 5	true
!=	Not equal to	5 != 3	true
===	Strict equal to	5 === '5'	false
!==	Strict not equal to	5 !== '5'	true
>	Greater than	5 > 3	true
<	Less than	5 < 3	false
>=	Greater than or equal to	5 >= 5	true
<=	Less than or equal to	5 <= 3	false

4. Logical Operators (for combining conditions)

Operator	Name	Example	Result
&&	Logical AND	true && false	false
	Logical OR	true false	true
!	Logical NOT	!true	false

Conditional Statements (if, else, switch)

Conditional statements are used to execute different blocks of code based on different conditions.

JavaScript provides three main conditional statements:

if, **else if**, **else**, and **switch**.

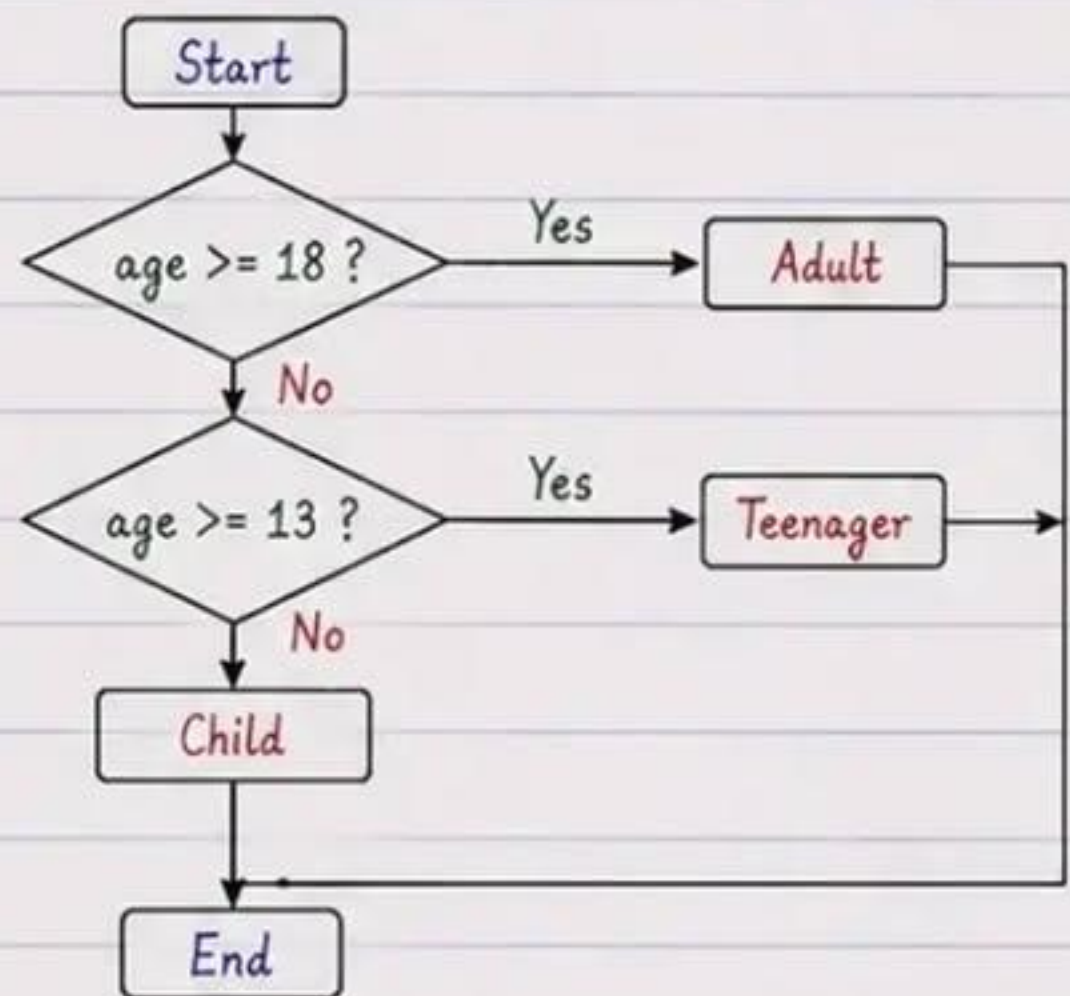
They help our program make decisions.

1. if, else if, else Statement

- Used to execute one block of code among multiple conditions.
- Conditions are checked from top to bottom.

Example:

```
let age = 18;
if (age >= 18) {
  console.log("You are an adult");
} else if (age >= 13) {
  console.log("You are a teenager");
} else {
  console.log("You are a child");
}
```

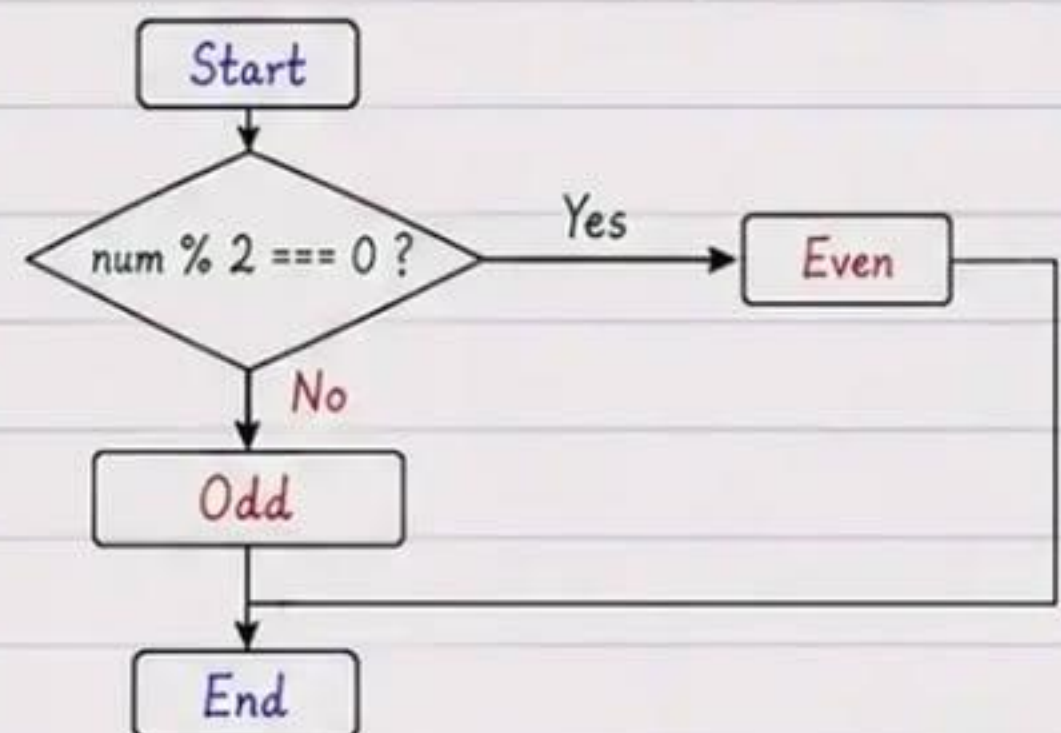


2. if, else Statement

- Used when we have only two options.
- If the condition is true, **if** block runs, otherwise **else** block runs.

Example:

```
let num = 10;
if (num % 2 === 0) {
  console.log("Even Number");
} else {
  console.log("Odd Number");
}
```

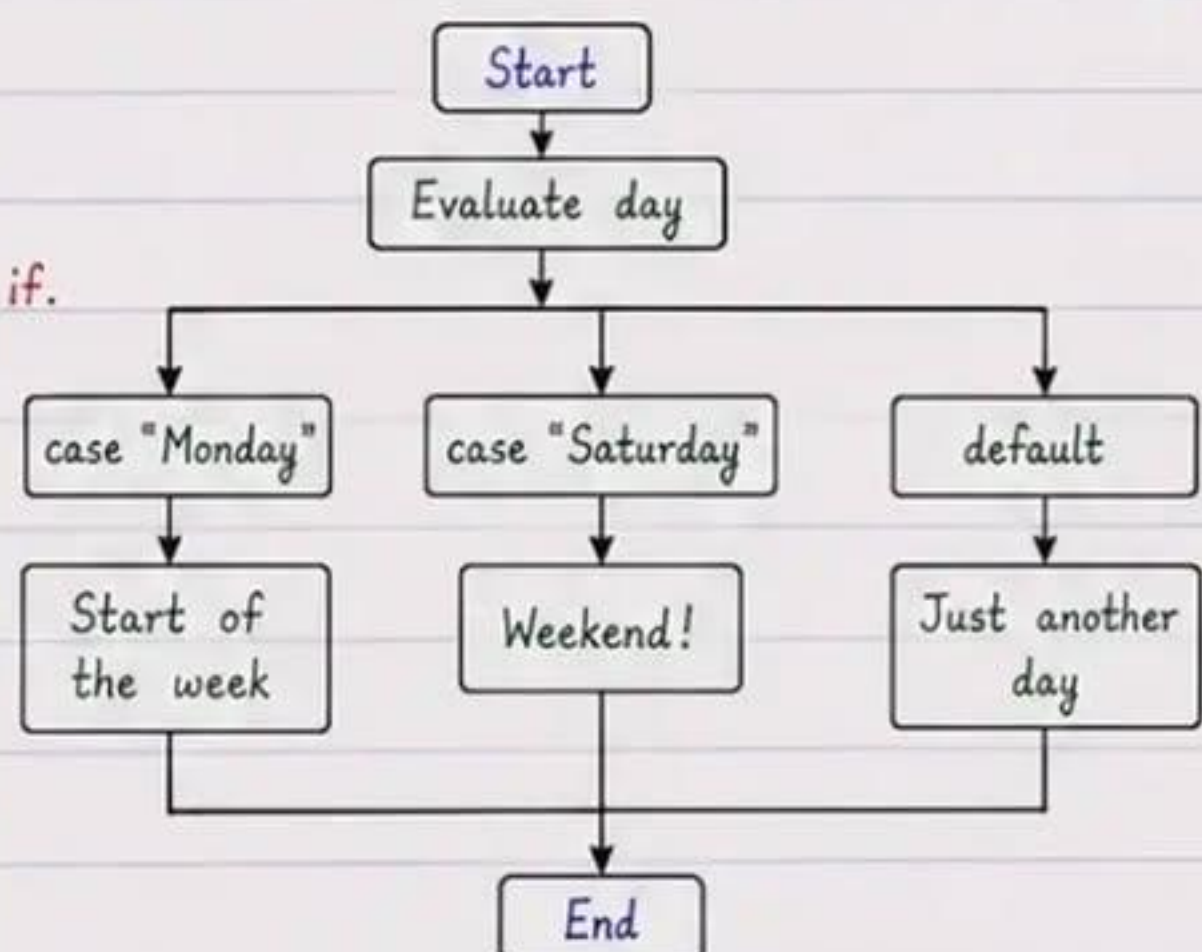


3. switch Statement

- Used when we have many possible values to compare with a single variable.
- It is more readable than multiple **if...else if**.

Example:

```
let day = "Monday";
switch (day) {
  case "Monday":
    console.log("Start of the week");
    break;
  case "Saturday":
    console.log("Weekend!");
    break;
  default:
    console.log("Just another day");
}
```



Loops (for, while)

Loops are used to run a block of code repeatedly as long as a condition is true.

JavaScript has two commonly used loops: **for** and **while**.

1. for Loop

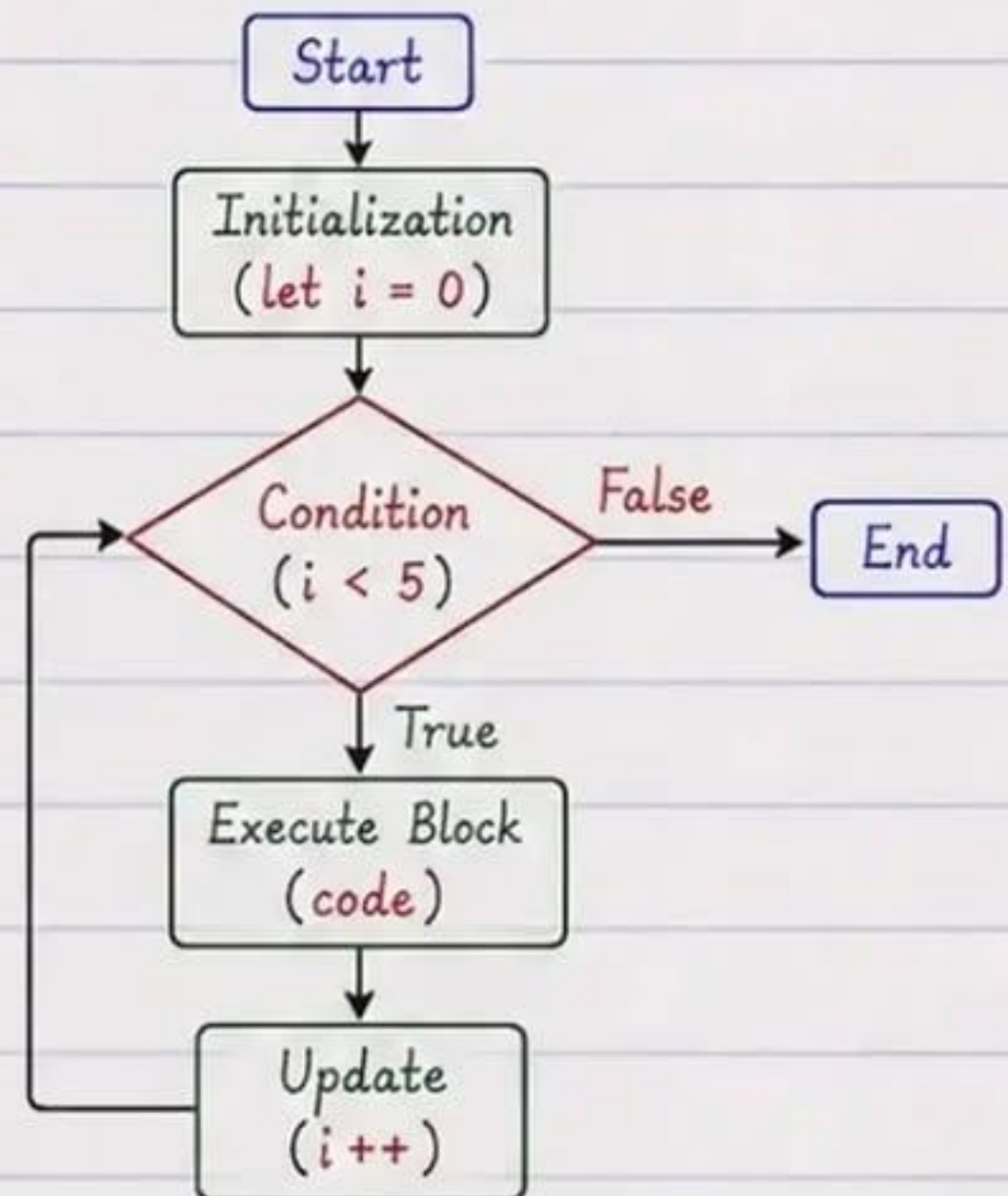
- Used when we know how many times we want to repeat.
- It has three parts: initialization, condition, and update.

Syntax:

```
for (initialization; condition; update) {  
    // code to be executed  
}
```

Example:

```
for (let i = 1; i <= 5; i++) {  
    console.log(i); // 1, 2, 3, 4, 5  
}
```



2. while Loop

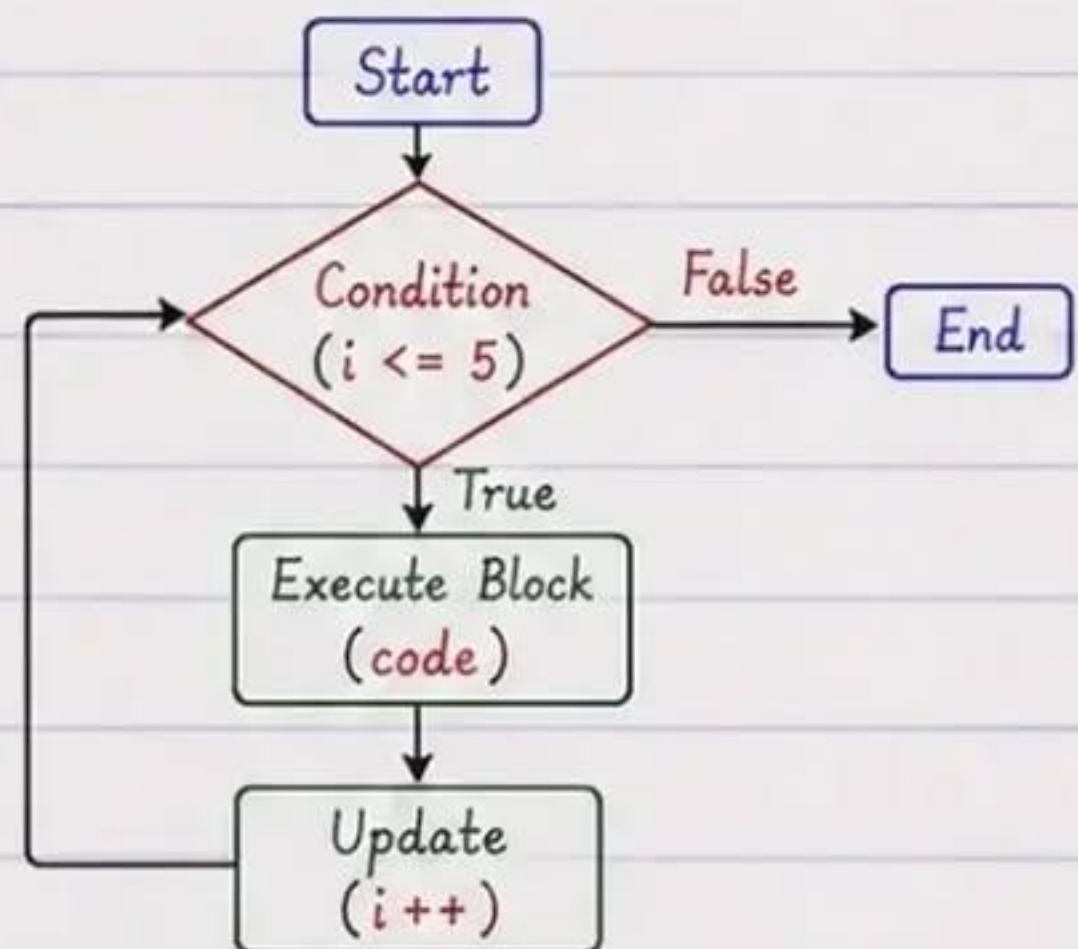
- Used when we don't know how many times to repeat.
- The block will run as long as the condition is **true**.
- We must update the condition inside the loop, otherwise it may run forever.

Syntax:

```
while (condition) {  
    // code to be executed  
}
```

Example:

```
let i = 1; // initialization  
while (i <= 5) {  
    console.log(i); // 1, 2, 3, 4, 5  
    i++; // update  
}
```



Loops (do...while)

The do...while loop is used to execute a block of code repeatedly as long as a condition is true.

The main difference from while loop is that in do...while loop, the code block is executed **at least once**, even if the condition is false from the beginning.

How it works:

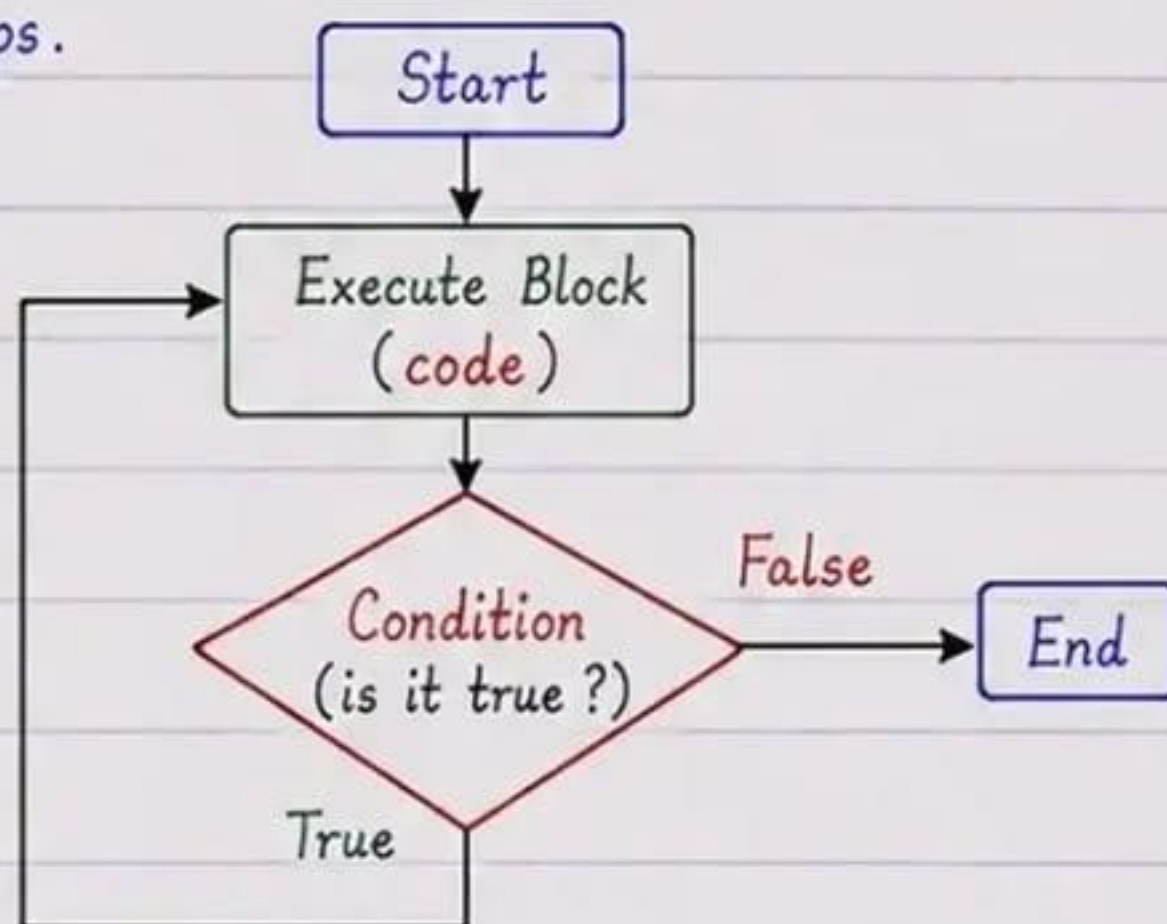
- First, the code block runs.
- Then the condition is checked.
- If the condition is **true**, the block runs again.
- If the condition is **false**, the loop stops.

Syntax:

```
do {  
    // code to be executed  
} while (condition);
```

Example:

```
let i = 1;    // initialization  
do {  
    console.log(i); // prints 1, 2, 3, 4, 5  
    i++;           // update  
} while (i <= 5);
```



Output:

1
2
3
4
5

- The code runs first, then the condition ($i \leq 5$) is checked.
- Since it is **true**, the loop runs again.
- When i becomes 6, the condition is **false**, and the loop stops.

Input & Output (prompt(), alert(), console.log())

JavaScript provides built-in methods to take input from the user and show output.

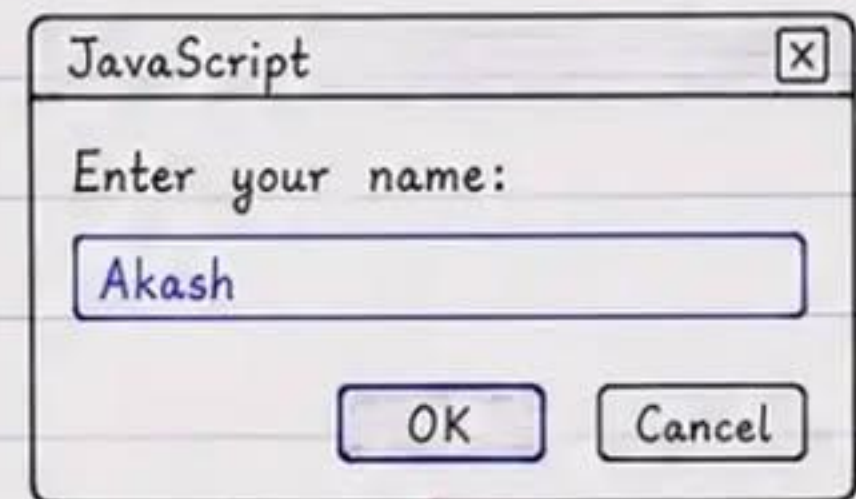
The most commonly used ones are `prompt()`, `alert()`, and `console.log()`.

1. `prompt()` - takes input from the user.

- It shows a dialog box with a message, an input field, and OK / Cancel buttons.
- It returns the text entered by the user.
- The return value is a **string**.

Example:

```
let name = prompt("Enter your name:");  
console.log("Hello, " + name + "!");
```



↓
User types "Akash"
and clicks OK.

2. `alert()` - shows an alert box with a message.

- It displays a message in a pop-up box with only an OK button.
- It **does not** return any value.

Example:

```
alert("Welcome to JavaScript!");
```



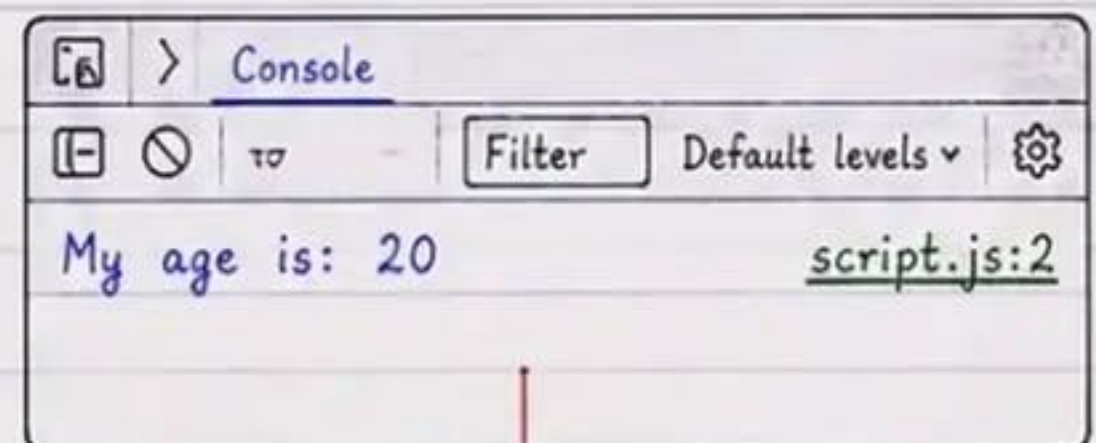
↓
Message is shown and user
clicks OK.

3. `console.log()` - displays output in the browser console.

- It prints messages, values, or results in the console (developer tools).
- Very useful for **debugging**.
- It **does not** show anything on the webpage.

Example:

```
let age = 20;  
console.log("My age is:", age);
```



↓
Output appears in the console,
not on the webpage.